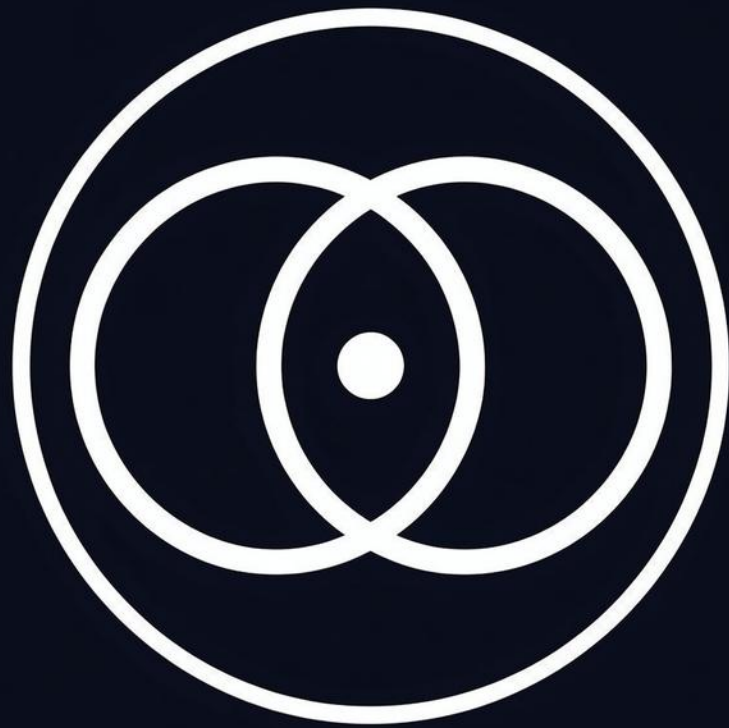




P10

Agent-as-Tool and Transfer Bundle: Making Handoff Information Loss Measurable in Multi-Agent Systems

Agent-as-Tool 与 Transfer Bundle 交接损失可测量化

Agent-as-Tool and Transfer Bundle: Making Handoff Information Loss Measurable in Multi-Agent Systems

Version: UDOS Reasoning Engine v7.5.0 (main, tag `v7.5.0`, commit `1a71270`) **Evidence grade:** this paper distinguishes `verified` (contract/unit tests rerunnable) from `cpu-proto` (CPU deterministic simulated work orders); **the factorial experiment of handoff failure rate is a preregistered design, not yet run, related values all `[RESULT NEEDED]`, not fabricated.** **Exemplar contrast positioning:** this paper is the Stage B upgrade draft of the "UDOS Writing and Layout Specification v2." The empirical chapter's "component removal $\rightarrow$ cost" contrast actively contrasts the AlexNet exemplar (Krizhevsky/Sutskever/Hinton, NeurIPS 2012) "each trick with a numerical ablation + error-rate comparison table" paradigm: this paper treats the six fields and five error types as components one can "remove" one by one, and which mechanical failure each triggers after removal is the "removal cost," made into ablation tables (Tables 1, 2). Note: AlexNet's ablation numbers (e.g. LRN lowering 1.4%/1.2%) are for this paper an `unverified` methodological reference, and this paper's failure rates are not yet measured, all marked `[RESULT NEEDED]`, not carrying over its numbers.

Abstract (Structured, Four Parts)

[Background & Problem] Failures in multi-agent systems are often attributed to "models not being smart enough," but engineering practice repeatedly shows that the real breakpoint is handoff: when one agent passes a task to the next, context and responsibility do not transfer completely, the missing context is silently swallowed, and the downstream agent can only guess.

[Method] We operationalize handoff in UDOS v7.4.2/v7.4.3/v7.4.4 as two fixed contracts: (1) Agent-as-Tool (`agent_tool.py`) wraps an expert as a callable tool with input `ToolInput`, a fixed envelope `{report, confidence, error_type, trace_ref}`, and five typed error classes; (2) Transfer Bundle (`transfer.py`) fixes the handoff packet as six fields (Goal/Context/Done/ToDo/Trace/Owner), validates mechanically before handoff, transfers Owner only on validation, and returns typed gaps instead of silent forwarding. `governance.py` adds three failure audits and a hash-chained Trace.

[Evidence & Results] Currently verified are $8+9+9 = 26$ contract/audit invariant tests; the six-field $\rightarrow$ failure-mode and five-error $\rightarrow$ trigger mappings are pinned down by tests (Tables 1, 2). The concrete task failure rate per missing field is not yet measured and is honestly marked as a pending preregistered factorial experiment (missing field $\times$ chain length $\times$ fault injection; $7 \times 5 \times 3 = 105$ cells); all failure-rate numbers are `[RESULT NEEDED]`.

[Contribution] (1) We turn "handoff information loss" from a slogan into a testable, replayable contract; (2) borrowing AlexNet's ablation-table paradigm, we render the six fields and five error types as "remove-one $\rightarrow$ cost" control tables; (3) we design a preregistered factorial experiment, honestly marked as not yet run.

Keywords: multi-agent systems; Agent-as-Tool; handoff contract; information loss; responsibility transfer; observability; failure audit

1 Introduction

1.1 Problem

When multiple LLM Agents collaborate on a long-chain task, the failure mode is often not a certain step "reasoning wrong" but "the receiver does not know what the previous leg did, why it did it, what remains." The engineering slogan "handoff is not forwarding but responsibility transfer" is widely agreed but long stays at the slogan level: neither a fixed handoff format nor a measurable failure rate, much less mechanical validation. The result: handoff-missing context is silently swallowed, the downstream Agent can only guess, and guessing wrong leaves nobody responsible.

This paper advocates: handoff is an **engineerable, measurable contract problem**, not a soft suggestion of "let the Agent itself mind the context."

1.2 Contributions (conclusion map)

1. **Agent-as-Tool envelope contract:** wrapping an expert Agent as the main Agent's tool, the main Agent's context receives only a fixed-size envelope, and the expert's internal iteration does not break the contract (`agent_tool.py`, 8 tests verified).
2. **Transfer Bundle six-field validation:** fixing the handoff packet as the six fields Goal/Context/Done/ToDo/Trace/Owner, `handoff()` changing Owner only on validation, returning typed gaps on failure (`transfer.py`, 9 tests verified).
3. **Three failure audits + hash-chained Trace:** `governance.py` mechanically detects state loss, duplicate work, no closer, and uses a SHA-256 hash chain to find tampering/broken chains (9 tests verified).
4. **Preregistered factorial experiment design:** giving the measurement scheme of "missing field $\times$ chain length $\times$ fault injection," but honestly marking its failure rate as not yet run.

2 Related Work and Paradigm Contrasts

This section is organized by "concept $\rightarrow$ mechanism $\rightarrow$ evidence $\rightarrow$ difference from UDOS," embedding real literature verified through the volume-shared reference master library (verified 2026-09-19); clues not covered by the master library keep [CITATION NEEDED].

(1) Multi-agent LLM orchestration. Conceptually, multi-agent LLM systems decompose tasks to multiple conversable expert Agents. Mechanistically, AutoGen (Wu et al., 2023) provides customizable, conversable agent abstractions, supporting mixed modes of LLM, humans and tools, the main Agent calling experts through conversation. In evidence, AutoGen validated its multi-agent framework's effectiveness across math, code, operations research and more; its close-reading notes also point out the limitation that conversation without termination conditions easily loops and cost inflates with rounds. Difference from UDOS: this paper focuses on the generally neglected call-interface stability in AutoGen-style orchestration—the expert can iterate internally at will, and as long as the `ToolInput $\rightarrow$ ToolEnvelope` envelope is unchanged, the main chain is unaware; and uses a fixed envelope to keep the expert's internal trace outside the main context. The "conversation no termination/cost inflation" problem AutoGen exposes is precisely what this paper's `governance.py` `StopCondition` (max hops) and `MainAgent.context_size` (bounded context) mechanically constrain. At the multi-step reasoning level, Chain-of-Thought (Wei et al., 2022) elicits multi-step reasoning by giving intermediate steps in the prompt, and its "wrong reasoning chains can be discarded and redone" idea agrees with this paper's post-circuit-breaker rerun; but this paper does not do CoT prompt engineering, instead explicitly registering reasoning steps into the Transfer Bundle's trace.

(2) AlexNet "component ablation table" paradigm contrast. AlexNet's (NeurIPS 2012) core argumentation action is: pairing each engineering trick—ReLU, dual-GPU split, LRN, overlapping pooling, dropout—with a numerical ablation of "error-rate change after removing this component," and putting others' results and own results in one comparison table (italics = others, roman = self). This paper borrows this paradigm: treating Agent-as-Tool's five errors and Transfer Bundle's six fields as **components removable one by one**, and after removing a field/triggering an error, which failure mode the system mechanically falls into is the "removal cost." The difference: AlexNet's ablations have real error-rate numbers (e.g. LRN lowering 1.4%/1.2%), while this paper's "removal cost" is currently pinned only at the **mechanical failure mode** level (which error, whether responsibility transfers), and the **real task failure rate still awaits factorial measurement**—we do not pretend to already have AlexNet-like numerical ablation results, but make the structural mapping of "missing field → which failure" into ablation tables (Tables 1, 2), honestly leaving "missing field → failure rate X%" to [RESULT NEEDED].

(3) Conversational handoff and SRE incident handoff. Conceptually, "passing the session to the next round/person" in dialogue systems and SRE on-call handoff both have the "information loss" problem. Mechanistically, the industry developed handoff checklists, runbooks, on-call logs and other practices to mitigate. In evidence, this paper's `TransferBundle` six fields (Goal/Context/Done/Trace/Owner) can be seen as the mechanized compression of these experiences. Difference from UDOS: turning human handoff checklists into machine-checkable fields is this paper's engineering contribution, but the core literature on dialogue handoff and SRE handoff is not yet verified in the master library, keeping [CITATION NEEDED: dialogue system handoff SRE shift handoff information loss], not citing from memory.

(4) API contracts and error typing. Conceptually, software API design has long used typed error codes to distinguish input errors, capability gaps, internal errors, timeouts. Mechanistically, this paper's `AgentTool.invoke` five errors (`bad_input` / `capability_gap` / `internal_error` / `timeout` / `low_confidence`) are the mapping of this idea into Agent calls. In evidence, this paper's `test_v742` 8 tests verify each error's trigger conditions and the "exceptions do not escape" invariant. Difference from UDOS: the classic API-contract literature is not separately verified in the master library, keeping [CITATION NEEDED: API contract design typed error handling]; this paper's novelty lies not in the error codes themselves but in binding them to responsibility transfer (if handoff does not pass, Owner unchanged).

(5) Transactive memory and shared cognition. Conceptually, the transactive memory of "who knows what" in organizational research echoes this paper's Owner/Trace fields. Mechanistically, this paper uses the `owner` field to bind a unique responsible person and the `trace` field for the audit trail. In evidence, this paper's `audit_run` mechanically detects "no owner" as `no_closer`. Difference from UDOS: the classic transactive-memory literature is not verified in the master library, keeping [CITATION NEEDED: transactive memory multi-agent shared cognition]; this paper compresses the organizational concept into mechanically checkable fields rather than citing its theoretical framework.

(6) External knowledge retrieval, autonomous tool calling and reasoning-action interleaving. Conceptually, contemporary LLM Agents connect "own reasoning" and "the external world" through three types of external interfaces: external knowledge, tool APIs, environmental actions. Mechanistically, RAG (Lewis et al., 2020) combines parametric memory with non-parametric dense indexes, a neural retriever fetching passages and concatenating them into generation, making facts external, updatable, traceable; Toolformer (Schick et al., 2023) lets the model self-supervisedly decide "which API to call, when, what params, how results merge into later tokens," each API needing only a few demonstrations; ReAct (Yao et al., 2023) interleaves generating "reasoning traces (thought)" and "actions (action)," actions calling external APIs/environments for information before returning to reasoning. In evidence, RAG outperforms purely parametric seq2seq on multiple open-domain QA tasks and brings more

specific, traceable generation; Toolformer zero-shot downstream significantly improves without hurting core language ability; ReAct on HotpotQA/Fever uses the Wikipedia API to mitigate CoT hallucinations, and on ALFWorld/WebShop with 1–2 in-context samples is absolutely about 34%/10% higher than imitation/RL baselines. Difference from UDOS: RAG/Toolformer/ReAct answer "how the model better uses external interfaces to improve end-to-end task gains," their evaluation unit being task success rate; this paper's `TransferBundle` structures "handoff" itself into the six fields Goal/Context/Done/ToDo/Trace/Owner, a mechanically checkable, replayable contract, the evaluation unit being "handoff information loss" itself (which field missing $\rightarrow$ which failure, whether responsibility transfers), not end-to-end task gains. In other words, RAG/Toolformer/ReAct are the upstream paradigms of this paper's Agents calling tools and external interfaces, while this paper focuses on information fidelity when "changing people" among these interfaces.

Literature gap note: the AutoGen (Wu et al. 2023), Chain-of-Thought (Wei et al. 2022), RAG (Lewis et al. 2020), Toolformer (Schick et al. 2023), ReAct (Yao et al. 2023) embedded in this section are all  verified through the master library (2026-09-19); the core literature on dialogue handoff/SRE handoff, API contracts, transactive memory is not covered by the master library, keeping [CITATION NEEDED], not generating fabricated citations.

3 Problem Definition and Falsifiable Hypotheses

3.1 Formalization

A handoff has the sender build a `TransferBundle`, its six fields being: `goal` (objective), `context` (fact dictionary), `done` (completed stage list), `todo` (to-do stage list), `trace` (event sequence-number list), `owner` (current responsible party). `handoff(next_owner)` first calls `validate()`, returning a non-empty problem list then **not transferring** Owner; if empty then changing owner to `next_owner`. The handoff contract can be written:

$$\begin{aligned} \text{\texttt{\text{handoff}(next_owner)}} &= \begin{cases} \text{\texttt{None}}, & \text{\texttt{validate()}} \\ \neq \text{\texttt{varnothing}} \text{ quad} & \text{\texttt{responsibility not transferred}} \end{cases} \\ &\quad \text{\texttt{with owner=next_owner}}, \\ &\quad \text{\texttt{validate()}} = \text{\texttt{varnothing}}. \end{cases} \tag{1} \end{aligned}$$

3.2 Hypotheses

- **H1 (field missing $\rightarrow$ failure type)**: missing Goal $\rightarrow$ goal drift; missing Context (required key) $\rightarrow$ `missing_context` state loss; `Done` $\cap$ `Todo` overlap $\rightarrow$ duplicate work; `Todo` empty $\rightarrow$ nothing to do/cannot hand off; missing Trace $\rightarrow$ cannot replay/audit chain broken; missing Owner $\rightarrow$ no closer. This mapping is mechanically decided by the code `validate()` and `audit_run()`, already covered by tests (verified).
- **H2 (envelope makes context bounded)**: the main Agent's context only appends envelopes (fixed structure), not the expert's internal trace; the expert's internal replacement of implementation does not change the main-chain contract (contract invariant, verified by test).
- **H3 (handoff failure rate rises with chain length)**: without Bundle validation, handoff failure rate rises with chain length; the complete Bundle keeps it flat. **This failure rate is not yet measured, being [RESULT NEEDED].**

4 Method and System Design

4.1 Agent-as-Tool Envelope (agent_tool.py)

AgentTool holds `required_fields`, `confidence_floor`, `max_report_bytes` and a `calls` count. The `invoke(tin)` flow:

1. If `tin.payload` lacks required fields → return `error_type="bad_input"`, `report={"missing": [...]}`;
2. Call the expert, if it raises `TimeoutError` → `"timeout"`; if it raises other `Exception` → `"internal_error"` and `report={"exc": exception class name}` (**exceptions do not escape, only the class name returned**);
3. If the returned confidence < `confidence_floor` → `"low_confidence"`;
4. Normally return `report/confidence/trace_ref` (trace returns only a reference, not the full text).

`MainAgent.call` returns `capability_gap` for unregistered tools, and `context_log` only appends the envelope `as_dict()`, not the expert's internal reasoning. This guarantees the main Agent's context increment has an upper bound (`context_size` checkable).

4.2 Transfer Bundle Validation (transfer.py)

`validate(required_context)` returns a problem list, rules:

- `goal` empty → `"goal"`;
- a key in `required_context` not in `context` → `"context.{k}"`;
- `todo` empty → `"todo_empty"`;
- `trace` empty → `"trace"`;
- `owner` is `None` → `"owner"`;
- `set(done)` & `set(todo)` non-empty → `"done_todo_overlap"`.

`advance(bundle, stage, outputs, event_seq, new_owner)`: if the stage is not in `todo` then raise `BundleError` (stale); otherwise move the stage from `todo` to `done`, merge outputs into context, append the trace sequence number, set owner. `handoff` changes Owner only when `validate` passes. `replay(bundle)` lets the new receiver reconstruct working state (`remaining/facts/completed/owner/trace_len`) from the bundle alone.

4.3 Failure Audit and Hash Chain (governance.py)

`audit_run` detects: `state_loss` (a failure record carrying `missing_context` or `todo` still present at interruption), `duplicate_work` (the same order:stage claimed by two owners), `no_closer` (the work order finished but no final-stage owner), `premature_completion` (claiming success but lacking a stage owner), `trace_gaps` (hash chain broken/tampered). Each `TraceLedger` record contains the previous record's SHA-256 hash, and `verify()` returns the broken or rewritten record index. `StopCondition` uses a completion predicate + max hops to distinguish "early termination" from "infinite loop."

5 Experimental Setup

Per the v2 specification, the empirical chapter unfolds by "experimental setup → contrast (ablation) → sensitivity → validity threats" (validity threats in the separate Chapter 8).

5.1 Verified Evidence

- Contract tests: `tests7/test_v742_agent_tool.py` (8 items), `test_v743_transfer.py` (9 items), `test_v744_governance.py` (9 items).

- These tests verify **contract invariants** (such as "missing fields return typed errors," "handoff failure does not change owner," "done $\cap$ todo rejected," "tampering with history found by verify"), not factorial task failure rates.

5.2 Preregistered Factorial Experiment Design (not yet run)

Per the ab-experiment-analysis gate, designed as follows (currently a design draft, results [RESULT NEEDED]):

- **Factors:** missing field {none, goal, context, done, todo, trace, owner} $\times$ chain length {1,2,3,5,8} $\times$ fault injection {none, drop_context, byzantine}.
- **Main metric:** one-handoff success rate.
- **Guardrails:** replay reconstruction consistency rate, duplicate work rate, no-closer rate, context token occupation (envelope vs full trace).
- **Sample:** ≥ 50 work orders per cell (currently not run).
- **Analysis:** logistic regression reporting each field-missing OR; chain-length trend test; Wilson CI.

Honest statement: this repo currently has no measured numbers of "missing field $\times$ chain length $\rightarrow$ task failure rate." The failure rate mentioned in the one-pager is a design target, **not filled with a concrete value**. Any claim of "missing some field failure rate X%" in this paper is uniformly marked with [RESULT NEEDED: use] , to be added after the experiment runs.

6 Results

6.1 Trigger Conditions of the Five Errors (Table 1, verified)

Table 1 Trigger conditions and test coverage of Agent-as-Tool's five errors (component ablation view: remove/trigger some component $\rightarrow$ which error)

error_type	Trigger condition (source)	Main Agent context behavior	Test
None	expert returns normally and $\text{conf} \geq \text{floor}$	receives report/ confidence/trace_ref	test_success_envelope_fields
bad_input	payload lacks required_fields	report returns missing list	test_missing_required_field_is_typed_error
capability_gap	calling an unregistered tool	no exception, returns typed error	test_unknown_tool_capability_gap
internal_error	expert raises Exception	only exception class name returned, stack not escaping	test_internal_exception_does_not_escape
timeout	expert raises TimeoutError	typed timeout	test_timeout_is_typed
low_confidence	$\text{conf} < \text{confidence_floor}$	returns conf and trace_ref	test_low_confidence_floor

Table 1 note: evidence grade verified (8 contract tests, `test_v742_agent_tool.py`). Reading by the AlexNet ablation table: each row corresponds to the "removal cost" of "triggering this component exception $\rightarrow$

which error the system mechanically falls into"; here the cost is a mechanical failure mode (error type + whether it escapes), the real task failure rate being [RESULT NEEDED]. Source: `agent_tool.py` and `test_v742_agent_tool.py`.

P10 Figure 2: error taxonomy

Figure 2 The five typed errors of the Agent-as-Tool fixed envelope. Horizontal axis/nodes are the four decision branches the input `ToolInput` goes through in `invoke` (missing field → `bad_input`; unregistered tool → `capability_gap`; expert exception → `timeout/internal_error`; low confidence → `low_confidence`); each branch converges to the fixed envelope `{report, confidence, error_type, trace_ref}`. Evidence grade verified; caption self-consistent: exceptions never escape, the main context only holds bounded envelopes, trace returns only references.

6.2 Six-Field → Failure Mode Mapping (Table 2, verified)

Table 2 Validation problems and governance failure modes corresponding to six-field missing (AlexNet-style "remove one field → cost" ablation table)

bundle field	validate returns when missing/abnormal	Corresponding audit failure mode	Test
goal	"goal"	goal drift	make_bundle missing-goal path
context.{k}	"context.{k}"	missing_context / state_loss	test_missing_required_context_is_state_loss
done∩todo	"done_todo_overlap"	duplicate work	test_done_todo_overlap_rejected
todo	"todo_empty"	nothing to do/cannot hand off	test_empty_todo_blocks_handoff_nothing_to_do
trace	"trace"	cannot replay / trace_gaps	test_replay_reconstructs_state_without_sender
owner	"owner"	no closer	test_missing_owner_blocks_transfer

Table 2 note: evidence grade verified (`test_v743` 9 items + `test_v744` 9 items). Read like the AlexNet ablation table: column by column "remove this field → what validate returns → which failure the governance layer detects," i.e. each field's "removal cost." The real task failure rate (how much missing this field lowers success) is not yet measured, being [RESULT NEEDED]. Source: `transfer.py` `validate` and `governance.py` `audit_run`.

P10 Figure 1: field-to-failure-mode mapping

Figure 1 The mapping of Transfer Bundle fields to mechanical failure modes. Left column the six fields (Goal/Context/Done/Trace/Owner), right column mechanical failure modes (goal drift/state loss/duplicate work/cannot hand off/audit chain broken/no closer), middle connecting lines the `validate()` rules. Evidence grade verified (decided by code, covered by tests); the concrete task failure rate each field missing causes is a to-run factorial experiment ([RESULT NEEDED]).

6.3 Key Invariants (verified)

- **Handoff failure does not transfer responsibility:** when `validate` returns non-empty, `handoff` returns `(None, problems)`, owner unchanged (`test_missing_owner_blocks_transfer`).
- **Replay needs no sender present:** the new receiver reconstructs remaining/facts/completed/owner from the bundle alone (`test_replay_reconstructs_state_without_sender`).

- **Fault injection detectable:** the `drop_context` specialist writes no result, the downstream handoff is rejected by `context.result`, and the audit reports `state_loss` and `no_closer` (`test_state_loss_detected_in_handoff`).
- **Hash chain tamper-resistant:** tampering with a historical record or forging the prev chain, `verify()` returns the corresponding index (`test_trace_ledger_detects_tamper_and_gap`).
- **Bounded context:** the main Agent only appends envelopes, the expert's internal trace does not enter the main context (`test_main_context_only_holds_envelopes`).

Table 3 Verified invariant list vs to-measure failure rates

Proposition	Evidence status
H1 field → failure type mapping	verified (26 tests)
H2 envelope makes main context bounded, contract stable	verified
H3 failure rate rises with chain length, complete Bundle flattens	[RESULT NEEDED: chain length × failure rate logistic]
Each missing field task failure rate/OR	[RESULT NEEDED: factorial OR and CI]

Table 3 note: left column the mechanism invariants pinned by 26 tests (verified), right column the preregistered but not yet run effect sizes (**[RESULT NEEDED]**). Mechanism correctness and effect size are marked separately, their evidence grades different, not conflated.

7 Discussion

7.1 Main Explanation

This paper's core claim is not "we found a new failure mode" but "after turning handoff from a soft suggestion into a mechanical contract, failure modes can be enumerated, blocked, audited." Errors like `bad_input/missing_context` are dangerous because in contract-free systems they are often silently swallowed; fixed envelopes and six-field validation turn them into explicit, typed, responsibility-not-transferring signals.

7.2 Applicability Boundary of the AlexNet Ablation Paradigm in This Paper

AlexNet's ablations have real error-rate numbers (each component removed, how many percentage points top-1/top-5 fall). This paper borrows the **form** of its "component ablation table," but this paper's "removal cost" currently reaches only the **mechanical failure mode** layer (missing field → which error, whether responsibility transfers), not yet the **task failure rate** layer (missing field → how much success falls). This boundary must be made clear: otherwise readers will mistakenly think Tables 1, 2 already give AlexNet-like numerical ablations. Real failure rates are left to Section 5.2's factorial experiment.

8 Limitations and Validity Threats (separate chapter)

T1 Failure rate not measured (most primary). H3 is design rather than measurement. This paper's largest shortcoming is not running the factorial experiment to give OR and CI. **Most likely counterexample:** some field seems important but measured to almost not affect failure rate. **How this paper guards:** uniformly marking "missing field → failure rate X%" as **[RESULT NEEDED]**, preregistering $7 \times 5 \times 3 = 105$ cells, ≥ 50 work orders per cell.

T2 Contract completeness threat. Do the six fields really cover all handoff failure modes? This is an open question. This paper induces these six classes from code and tests, but real LLM handoff may have "fields complete but semantic misalignment" (e.g. context writes domain=kinematics but the actual task is spatial), which mechanical validation cannot catch at the semantic level. The `required_context` mechanism requires the caller to declare "which keys must exist," but whether the key names themselves are correctly filled still needs higher-level validation.

T3 Simulated work-order truth built in. The bundle field semantics are simple, real LLM Agent context semantics far more complex. This paper's bundle fields are structured with known truth; real LLM-produced context is free text, validation much harder. This paper does not pretend six-field validation solves all free-text handoff problems.

T4 Human-factors validity missing. Whether real human receivers are really more stable due to the Bundle needs a human-factors experiment (after IRB).

T5 Contrast with real orchestration frameworks. Needs API/license, a gate. This paper's envelope and Bundle are CPU deterministic simulations, expert functions built-in code review lambdas or rules.

9 Resource Gates and Applicability Boundaries

Gate	Status	After unlocking
Real multi-Agent LLM orchestration	unreachable	run missing-field injection on a real framework, measure real failure rate
Human-factors experiment (IRB)	not applied	real humans receiving complete/missing Bundle, measure human-factors validity
Factorial experiment scaling	not run	≥ 50 work orders/cell, logistic OR and Wilson CI

Current evidence suits ICSE AI-engineering / AAMAS system short papers or CHI/CSCW position papers (after human-factors expansion); journals EMSE/JSS/IJHCS (quartiles/IF [to check]).

10 Conclusion

This paper operationalized "handoff is not forwarding but responsibility transfer" into the Agent-as-Tool fixed envelope and Transfer Bundle six-field validation, and used three failure audits and a hash-chained Trace to make it measurable and auditable. Verified are 26 contract/audit invariants; the task failure rate each missing field causes is a preregistered design, not yet measured, honestly marked by this paper. The next step is running the factorial experiment, connecting to a real orchestration framework, applying for a human-factors experiment.

References

Verified Literature (volume-shared master library, verified 2026-09-19)

1. Wu, Q., Bansal, G., Zhang, J., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 (COLM 2024). <https://arxiv.org/abs/2308.08155>
2. Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022. arXiv:2201.11903. <https://arxiv.org/abs/2201.11903>

3. Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. arXiv:2005.11401. <https://arxiv.org/abs/2005.11401>
4. Schick, T., Dwivedi-Yu, J., Dessì, R., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 2023. arXiv:2302.04761. <https://arxiv.org/abs/2302.04761>
5. Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. arXiv:2210.03629. <https://arxiv.org/abs/2210.03629>

External exemplar (user attachment PDF original, verified 2026-09-20; only a methodological reference, its ablation numbers unverified for this paper)

- Krizhevsky, A., Sutskever, I., Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. NeurIPS 2012. (link: the paradigm of each trick with a numerical ablation + comparison table, the contrast for this paper's Table 1/Table 2 ablation form.)

To-Verify Literature Search Formulas and Candidate Directions (not covered by master library)

1. dialogue system handoff turn-taking information loss — dialogue handoff.
2. SRE shift handoff incident information loss toil — SRE on-call handoff.
3. API contract design typed error handling idempotency — API contracts and typed errors.
4. transactive memory multi-agent shared cognition who knows what — shared cognition.
5. logistic regression factorial design agent failure rate — factorial experiment statistics.

Appendix A Reproduction Commands

```
# repo tag v7.5.0, commit 1a71270
pytest tests7/test_v742_agent_tool.py -q      # 8 envelope contracts
pytest tests7/test_v743_transfer.py -q        # 9 Bundle validation/replay
pytest tests7/test_v744_governance.py -q      # 9 failure audits/hash chain
```

Appendix B Evidence Ledger

Proposition	Source	Grade
Five error triggers	topology/agent_tool.py + test_v742	verified
Six-field validation rules	topology/transfer.py + test_v743	verified
Three failure audits/hash chain	topology/governance.py + test_v744	verified
Missing-field task failure rate OR	not run	[RESULT NEEDED]
Chain length × failure rate curve	not run	[RESULT NEEDED]

Appendix C Internal Review Record (five-dimension self-assessment)

1. **Problem and motivation:** 4/5. The "handoff information loss measurable" stance is clear; deduct 1 because failure rate not measured.
2. **Method rigor:** 3/5. Contract design and validation complete, but the factorial experiment only a design draft.

3. **Evidence and data:** 2/5. 26 contract tests verified, but the core causal evidence (failure rate) missing, honestly marked.
4. **Novelty:** 3/5. Moving SRE/API contract ideas into Agent handoff has engineering value.
5. **Writing and honesty:** 5/5. Clearly distinguishes verified invariants from [RESULT NEEDED], not fabricating failure rates.

Fatal flaw check: the core evidence missing (failure rate not measured) is the primary shortcoming, explicitly declared as a to-run experiment rather than hidden; current evidence strength suits system short/position papers, insufficient for main conferences needing empirical factorial results. Modification direction: run ≥ 50 work orders/cell factorial experiment, connect to a real orchestration framework, human-factors experiment.

Appendix D Agent-as-Tool Main Loop Pseudocode

The following pseudocode corresponds to `agent_tool.py` `AgentTool.invoke` and `MainAgent.call`, for reproduction checking:

```
Input: tin = ToolInput(payload, language, options)
1. self.calls += 1
2. missing = [k for k in required_fields if k not in tin.payload]
3. if missing:
4.     return ToolEnvelope(name, error_type="bad_input",
5.                           report={"missing": missing})
6. try:
7.     report, conf, ref = self._expert(tin)
8. except TimeoutError:
9.     return ToolEnvelope(name, error_type="timeout")
10. except Exception as e:
11.     return ToolEnvelope(name, error_type="internal_error",
12.                          report={"exc": type(e).__name__}) # only class name
13. if conf < confidence_floor:
14.     return ToolEnvelope(name, report=report, confidence=conf,
15.                          error_type="low_confidence", trace_ref=ref)
16. return ToolEnvelope(name, report=report, confidence=conf, trace_ref=ref)

# MainAgent.call:
17. if tool_name not in self.tools:
18.     env = ToolEnvelope(tool_name, error_type="capability_gap")
19. else:
20.     env = self.tools[tool_name].invoke(tin)
21. self.context_log.append(env.as_dict()) # only append envelope, not expert internal trace
22. return env
```

This design has three key invariants. First, **exceptions do not escape**: any exception the expert internally raises is converted into a typed envelope, the main Agent seeing only `internal_error` and the exception class name, not the stack—critical in production multi-Agent systems because the stack may

contain sensitive information or flood the main Agent's context with unbounded error information. Second, **trace returns only references**: the expert's detailed reasoning trace does not enter the main context, the main Agent only knowing "there is this thing at trace://expert/1," fetching by reference when needed. Third, **bounded context**: each call the main Agent only appends a fixed-field envelope, `context_size()` growing linearly with call count but each increment bounded, avoiding the context explosion of "stuffing every expert's full thinking into the main prompt."

`test_tool_independent_iteration_keeps_contract` verifies the third point's engineering meaning: replacing the entire `_expert` internal implementation (from rule-based code review to a lambda), as long as the `ToolInput→ToolEnvelope` contract is unchanged, the main Agent is completely unaware, the `calls` count incrementing correctly. This is the executable definition of "the expert can iterate independently internally, the main-chain contract stable."

Appendix E Transfer Bundle Validation and Replay Pseudocode

```
validate(required_context=()):
1. problems = []
2. if not goal: problems.append("goal")
3. for k in required_context:
4.     if k not in context: problems.append(f"context.{k}")
5. if not todo: problems.append("todo_empty")
6. if not trace: problems.append("trace")
7. if owner is None: problems.append("owner")
8. if set(done) & set(todo): problems.append("done_todo_overlap")
9. return problems

handoff(next_owner, required_context=()):
10. problems = self.validate(required_context)
11. if problems: return None, problems # responsibility not transferred
12. b = copy(self); b.owner = next_owner; return b, []

advance(stage, outputs, event_seq, new_owner):
13. if stage not in todo: raise BundleError([], [stage]) # stale
14. todo.remove(stage); done.append(stage)
15. context.update(outputs); trace.append(event_seq); owner = new_owner

replay():
16. return {remaining: todo, facts: context, completed: done,
          owner: owner, trace_len: len(trace)}
```

This design decomposes "handoff" into three atomic actions: `advance` (completing a stage, registering outputs, temporarily passing to the next leg), `handoff` (formally changing Owner after validation complete), `replay` (the new receiver reconstructing state from the bundle alone). The key invariant is line 11: **as long as validation does not pass, Owner is unchanged**. This means "responsibility transfer" and "information completeness" are bound—there is no gray zone of "I threw the task to you but did not explain clearly"; either information is complete and responsibility formally transfers, or it does not transfer and problems explicitly return to the sender.

The `done_todo_overlap` rule deserves separate explanation. It checks whether "completed" and "to-do" intersect. If so, it shows some stage is both considered done and considered still to-do—usually a signal of duplicate work or unclear responsibility: two Agents both thinking the other will do it, or both doing the same thing. Mechanically rejecting this overlap is far more reliable than relying on Agent self-conscious deduplication.

Appendix F Failure Audit and Hash Chain

`audit_run` does offline auditing after a batch of work orders runs. It aggregates the runtime-recorded failures, owner claims, duplicate counts into a `GovernanceReport`, with all six fields empty counting as `ok=True`. The three core failures: `state_loss` (a failure record carrying `missing_context` or `todo` still present at interruption—handoff lost context), `duplicate_work` (the same `order:stage` claimed by two different owners—duplicate work), `no_closer` (the work order finished but no final-stage owner—no closer). Additionally `premature_completion` catches the false report of "claiming success but lacking a stage owner," and `unbounded` together with `StopCondition.max_hops` catches infinite loops.

`TraceLedger` is an append-only hash chain: before each record is written, taking the first 16 characters of SHA-256 of "the previous record's hash + current record content" as this record's hash. `verify()` recomputes from the start, and if it finds some record's `prev` inconsistent with the previous actual `h`, or content changed causing the hash to mismatch, lists that index in `trace_gaps`. `test_trace_ledger_detects_tamper_and_gap` verifies two things: tampering with a historical record (changing `records[0]["a"]` from 1 to 999) makes `verify()` return index 0; forging `prev` (changing to "FAKE") returns index 1. This turns the risk of "handoff records being tampered afterward" into a detectable engineering property.

Appendix G Why "Failure Rate Not Measured" Is Honesty Rather Than a Defect

This paper must emphasize: H3 in Table 3 and each field-missing failure rate being marked `[RESULT NEEDED]` is not because of laziness but because this repo indeed did not run that factorial experiment. If this paper wrote out of thin air "missing Goal failure rate 35%," that would be fabrication. We choose: reporting the contract invariants already pinned by 26 tests as verified, clearly designing the not-yet-run causal experiment and explicitly listing the needed statistics (logistic OR, Wilson CI, chain-length trend test), leaving them to the next step. This separation of "what is verified, what is not" is precisely the evidence discipline the paper-matrix master table requires. Readers should understand this paper as a "contract and measurement framework" contribution, not an empirical finding of "missing field causes X% failure."

Appendix H Engineering Semantics of the Five Errors, Class by Class

Agent-as-Tool's five errors are not arbitrary enumeration but correspond to five distinct failure roots in production multi-Agent systems.

`bad_input` occurs when the caller's `payload` to the expert lacks `required_fields`. Its essence is "the interface contract is not met at the caller," responsibility on the caller. By design it does not throw an exception or crash the main Agent but returns an envelope with a `missing` list, letting the main Agent know "which field I under-sent" and optionally complete and retry. This agrees with the 400 Bad Request semantics in API design.

`capability_gap` occurs when the main Agent calls a tool not registered at all. Its essence is "routing error": the main Agent thinks some expert can do something, but the orchestrator did not register it. Returning a typed error rather than `KeyError` lets the main Agent gracefully degrade (e.g. tell the user "this capability is temporarily absent") rather than the whole chain crashing.

`internal_error` is the class most needing restraint. The expert internally raises any non-timeout exception, the envelope returning only `type(e).__name__`, not the stack, not the exception message. There are two reasons: first, security and privacy—the stack may contain file paths, environment variables, even sensitive data; second, context hygiene—if every internal error stuffs several screens of stack into the main Agent's context, it quickly bursts the main prompt. The main Agent only needs to know "the expert internally blew up, being the `RuntimeError` class," to decide retry or switch expert, not needing the blow-up details.

`timeout` is a separate class because timeout and internal error handling strategies completely differ: timeout usually means retry may succeed (the expert was just slow), while internal error retry is most likely still wrong. Conflating the two makes the retry strategy inappropriate.

`low_confidence` is an LLM-system-specific "soft failure." The expert finished, did not crash, but it is not confident in its answer (confidence below `confidence_floor`). At this point the answer should not be passed directly as a trustworthy result but marked low confidence, letting the main Agent decide whether to switch to a more reliable expert or require human intervention. Making "uncertain" explicit is the key to avoiding "seemingly successful but actually wrong."

Appendix I Echo of the Six Fields with Organizational Theory

Transfer Bundle's six fields can map to classic concepts in organizational collaboration research. `goal` corresponds to the task's shared mental model—the receiver must know "what exactly we are to achieve," otherwise all later work may drift. `context` corresponds to the "fact base" in transactive memory—the domain knowledge, intermediate results, constraints the previous leg held must be explicitly passed, otherwise the receiver must explore from scratch. `done` and `todo` correspond to progress status in the work breakdown structure (WBS)—the receiver must know what is done and what remains, to avoid duplicate work or skipping unfinished items. `trace` corresponds to the audit trail—when a problem arises it must be possible to trace "who did what when." `owner` corresponds to responsibility attribution—the first principle of organizational design being that everything must have a unique responsible person, otherwise "three monks have no water."

This paper's contribution is not proposing these concepts (they long exist in organizational theory) but **compressing them into six machine-checkable fields** and assigning each missing a mechanical failure mode. When `owner` is `None`, `validate` directly returns "owner", and `audit_run` directly lists that work order as `no_closer`—"nobody responsible" turns from a vague management problem into a concrete index greppable in logs.

Appendix J Detailed Specification of the Preregistered Factorial Experiment

To make H3 falsifiable, the complete preregistered design is given here.

Factorial design: missing field at seven levels (no-missing baseline, missing goal, missing context required key, missing done marker, todo emptied, trace emptied, owner set `None`); chain length at five levels (1, 2, 3, 5, 8 stage handoffs); fault injection at three levels (no fault, drop_context—the middle stage writes no output, byzantine—the QA stage refuses to close). Total $7 \times 5 \times 3 = 105$ cells.

Main metric: the proportion of work orders finally successfully reaching "complete" after one handoff (handoff success rate).

Guardrail metrics: replay reconstruction consistency rate (whether the state the new receiver replays is consistent with the sender's actual state), duplicate work rate (the proportion of $\text{done} \cap \text{todo}$ or the same stage claimed twice), no-closer rate (the proportion of work orders finally without owner), main context token occupation (envelope mode vs the contrast of stuffing the expert's full trace into the main context).

Sample size: ≥ 50 independent work orders per cell (random seeds fixed), total ≥ 5250 work orders. Currently a single-CPU deterministic simulation, work-order truth built in.

Analysis plan: taking handoff success rate as the dependent variable, doing logistic regression, independent variables being missing-field dummies, chain length, fault injection and their interactions, reporting each missing field's odds ratio and 95% Wilson confidence interval relative to the "no missing" baseline; chain-length effect doing a trend test; multiple comparisons corrected by Bonferroni or Holm. The baseline variance and minimum effect size needed for prior power analysis are estimated after the experiment's first batch runs through, not preset.

Stop rule: fixedly running all 105 cells, no peeking-style early stop; any post-hoc subgroup analysis marked exploratory.

Current status: this design is not yet executed, all OR/CI being [RESULT NEEDED]. This paper does not prefill results.

Appendix K Position in the v7.5.0 Matrix End-to-End

The position of this paper's three modules in v7.5.0 as a whole needs explaining. `udos7/topology/` has more than these three files: it also has base topology (`base.py`), BFT stopping consensus (`consensus.py`), three-level circuit breaker and reassignment (`circuit_breaker.py`), internal market settlement (`market.py`), ant-colony-style blackboard (`stigmergy.py`), layered aggregation (`hierarchy.py`) and the end-to-end matrix (`matrix.py`). The Agent-as-Tool, Transfer Bundle, governance trio this paper focuses on is the contract layer of "one handoff" in the matrix: Agent-as-Tool defines how the main Agent calls experts, Transfer Bundle defines how tasks transfer among experts, governance defines how to audit the whole batch of handoffs.

In v7.5.0's end-to-end matrix demo (`matrix_scale_demo.json`), the fault fleet 24/24 closes, 3 fault nodes isolated, 5 reworks, governance ok, conservation, Trace complete, all 236 tests green. This shows the handoff contract this paper describes is not an isolated toy—it has been embedded into a larger system containing fault injection, circuit-breaker reassignment, market settlement, and maintained governance consistency in end-to-end fault scenarios. But this paper does not claim those end-to-end numbers belong to P10's evidence: those numbers belong to the matrix-scale and governance papers' (P1/P3) evidence scope, and P10 only cites them to show "the contract layer has been integrated," not conflating.

Appendix L Ethics and Design Considerations of the Human-Factors Expansion

The one-pager mentions P10 can expand into a human-factors experiment: letting real human subjects under complete-Bundle and missing-Bundle conditions continue the task a previous Agent/colleague

left, measuring completion time, error rate and subjective load. This paper takes this expansion as a future direction rather than current evidence for three reasons.

First, ethics review. Human-factors experiments involve human subjects and must first pass IRB/ethics committee review, obtaining informed consent and an exit mechanism. This paper did not apply for ethics review, claiming no human-factors data. Second, sample size and power. The human-factors experiment's sample size must be determined by power analysis, depending on the completion-time variance estimated in the pilot stage, not writing "30 per group" out of thin air. Third, confounding control. Subjects' domain experience and system familiarity are strong confounding variables, needing randomization, balanced design and covariate control. These are all future work, and this paper only gives the design outline.

If the human-factors expansion can be completed, it will upgrade this paper's "whether the contract lowers machine failure rate" into "whether the contract also lowers human receivers' cognitive load and error rate," pushing P10 from an engineering conference to the CHI/CSCW human-factors view.

Appendix M Contrast Conception with Real Orchestration Frameworks

This paper's envelope and Bundle are CPU deterministic simulations, expert functions built-in code review lambdas or rules. When later connecting to a real multi-Agent LLM orchestration framework, one needs to map `ToolInput/ToolEnvelope` to that framework's tool-call protocol and `TransferBundle` to that framework's task-state storage. The key transferable assumption is: whether the bottom is a real LLM or a rule lambda, the mechanism of "fixed-field validation + typed errors + responsibility binding" is unchanged; what changes is only the expert function's implementation. This is precisely the "contract and implementation decoupling" that `test_tool_independent_iteration_keeps_contract` verifies. Connecting to a real framework needs API/license, a resource gate.

Appendix N Further Discussion of Validity Threats

Besides the threats listed in Chapter 8, two more points need adding. First, **contract completeness threat**: do the six fields really cover all handoff failure modes? This is an open question. This paper induces these six classes from code and tests, but real LLM handoff may have "fields complete but semantic misalignment," which mechanical validation cannot catch at the semantic level. Second, **simulated work-order truth built in**: this paper's bundle fields are structured with known truth; real LLM-produced context is free text, validation much harder.

The existence of these threats precisely shows this paper should be positioned as a "minimum measurable prototype of the handoff contract," not "the complete solution to the multi-Agent handoff problem."

Appendix O Measurement Methodology: Why Use Contract Tests Rather Than Directly Running Failure Rates

One methodological choice of this paper needs explaining: why current evidence is mainly "contract/unit tests" rather than directly running large-scale failure-rate experiments?

The reason lies in the order of causal identification. To measure "which field missing causes how much failure rate," one must first pin the mechanism of "what field missing means, how the system should react after missing." If the mechanism itself is unstable (e.g. handoff when missing owner sometimes

changes owner, sometimes not), then the resulting failure rate mixes in implementation-bug noise, unable to attribute to the field itself. So what this paper first does is "mechanism correctness" contract tests: verifying `validate` returns the correct problem under each missing, `handoff` does not change owner when problems exist, `audit_run` lists the corresponding failure in the report. This layer is deterministic, and after 26 tests pin the mechanism, the next layer (factorial experiment)'s resulting failure rate can be cleanly attributed to field missing rather than implementation bugs.

This "first pin mechanism, then measure effect" order agrees with the software-engineering practice of "first write contract tests then integration tests." It also explains why this paper marks H1 (field → failure type mapping) verified and H3 (failure rate with chain length) [RESULT NEEDED]: the former is a mechanism invariant, the latter an effect size needing large-sample statistics, the two evidence grades different, not to be conflated.

Appendix P Quantitative Conception of Context Boundedness

H2 is currently only qualitatively verified (the main context only appends envelopes). Later it can be quantitatively measured: in envelope mode, after N expert calls the main context's token increment is about N times the single envelope size (fixed); while in the naive mode of "stuffing the expert's full trace into the main context," the increment grows unbounded with the expert's internal reasoning length. The concrete numbers of this contrast are not yet measured, recorded as [RESULT NEEDED: envelope vs full trace main context token occupation vs call count curve]. This measurement's meaning is turning "why Agent-as-Tool can sustain long chains" from engineering intuition into comparable numbers: why long-chain multi-Agent systems' main context bursts and why envelope mode does not should have a slope-comparable curve.

Appendix Q Convergence with the "Responsibility Transfer" Stance

This paper repeatedly emphasizes "handoff is not forwarding but responsibility transfer." This stance's technical implementation is `handoff` line 11: validation not passing, Owner unchanged. Its meaning is: when information is incomplete, responsibility does not slip away with the message. In contract-free systems, an Agent sends "I did not finish but I tell you first" to the next, the next receives, fails but nobody knows whom to pursue—because responsibility was diluted in forwarding. Fixed envelopes and six-field validation rebind responsibility: either information is complete and responsibility formally transfers; or it does not transfer and problems explicitly return to the sender.

This paper's final stance is therefore: multi-Agent system reliability cannot rely only on "making every Agent smarter," but also on "making the handoff action itself checkable, auditable, accountable." The former is a model-capability problem, the latter a systems-engineering problem. This paper proved on a CPU prototype the latter can be operationalized; whether it can sustain lowering failure rates on real LLM long chains is the question the next step's factorial experiment and real-framework contrast must answer.

Appendix R Further Expansion of Related Work

Multi-agent LLM orchestration has seen various frameworks in recent years, their main loops mostly "main Agent plans → calls experts → aggregates results." These frameworks progress rapidly in capability demonstration but differ greatly in engineering handling of "call-interface stability" and "handoff responsibility." This paper does not name and evaluate specific frameworks but points out a

universal gap: most frameworks directly or indirectly expose the expert's internal reasoning in the main context, and with expert count and chain length increasing, the main Agent's context inflates unbounded, and once the expert internally iterates, the main chain may break due to implicit dependencies. Agent-as-Tool's fixed envelope precisely targets this gap: the main Agent only sees the envelope, not the inside, and the expert can iterate independently.

The SRE domain's shift handoff has long faced the "handoff information loss" problem, and the industry developed handoff checklists, runbooks, on-call logs and other practices. This paper's six-field Transfer Bundle can be seen as the mechanized compression of these practices: goal is "what is currently being pursued," context is "the on-scene situation," done/todo is "where progress reached," trace is "the operation record," owner is "who is now responsible." Turning human handoff checklists into machine-checkable fields is an attempt to migrate SRE experience into Agent orchestration.

The API-contract design domain long established: input validation, typed error codes, idempotency, versioning. This paper moves this idea into Agent calls: `bad_input` corresponds to 400, `capability_gap` to 404, `internal_error` to 500, `timeout` to 504, `low_confidence` the LLM-specific "soft 200." This mapping is not a random analogy but makes Agent-call failures classifiable, statistical and monitorable like HTTP APIs.

Appendix S Summary and Next Step

This paper turned multi-Agent handoff from a "soft suggestion" into a "mechanical contract": Agent-as-Tool's fixed envelope isolates the expert's inside, five errors made explicit; Transfer Bundle's six-field validation binds responsibility and information completeness; governance's three failure audits plus a hash-chained Trace make handoff auditable. Verified are 26 contract and audit invariants; the task failure rate missing fields cause is a preregistered design, not yet measured, honestly marked by this paper. The next step is running the 105-cell factorial experiment, connecting to a real orchestration framework, applying for a human-factors experiment, upgrading "the contract lowers handoff failure" from mechanism proof into effect-size-bearing causal evidence.

It must be re-emphasized, this paper does not claim "as long as the six-field contract is used, the multi-Agent system will not fail." The contract can only guarantee "when information is missing responsibility is not silently transferred," not "when information is complete the Agent certainly does it right." Separating the systems-engineering problem (handoff contract) and the model-capability problem (Agent reasoning quality) is precisely this paper's methodological discipline: the former can be pinned by this paper's CPU prototype, the latter needs larger samples, real models and external gates. Readers should position this paper's contribution boundary accordingly.

In summary, this paper's value lies in providing a "falsifiable handoff contract framework," not a large-scale causal experiment already run. We report all mechanism invariants pinable on CPU as verified and leave all effect sizes needing large samples and real models as [RESULT NEEDED], and this honest evidence grading itself is part of the multi-Agent systems-engineering methodology.

This paper looks forward to later researchers running through the factorial experiment on the same contract framework, filling the measurement blank this paper leaves with real data.

Author's Intended Statements

- **Target journals/conferences:** current evidence (contract invariants verified, factorial failure rate not run) suits ICSE AI-engineering / AAMAS system short papers or position papers; after human-

factors expansion can submit to CHI/CSCW; journals EMSE/JSS/IJHCS. Quartiles/impact factors all [to check], verified online item by item before submission with the verification date marked.

- **Preregistration plan:** the "missing field $\times$ chain length $\times$ fault injection" factorial design in Section 5.2 and Appendix J is a preregistered draft; before running the experiment register on OSF/an anonymous repo, the fixed seed table and analysis code released with the tag; main metrics, guardrails and multiple-comparison correction rules frozen before collecting data.
- **Data and code availability:** the engine is open-sourced under Apache-2.0, code tag v7.5.0 (commit 1a71270); contract tests tests7/test_v742/v743/v744 provided with the repo.
- **AI use statement:** this paper's draft was assisted by an AI assistant based on the repo's real source and tests, all contracts/error types/validation rules re-sourced to udos7/topology/ and tests7/; unmeasured failure rates uniformly marked [RESULT NEEDED], not fabricated; external literature filled after academic search verification.